Name: Kade McRae Mark _____/50

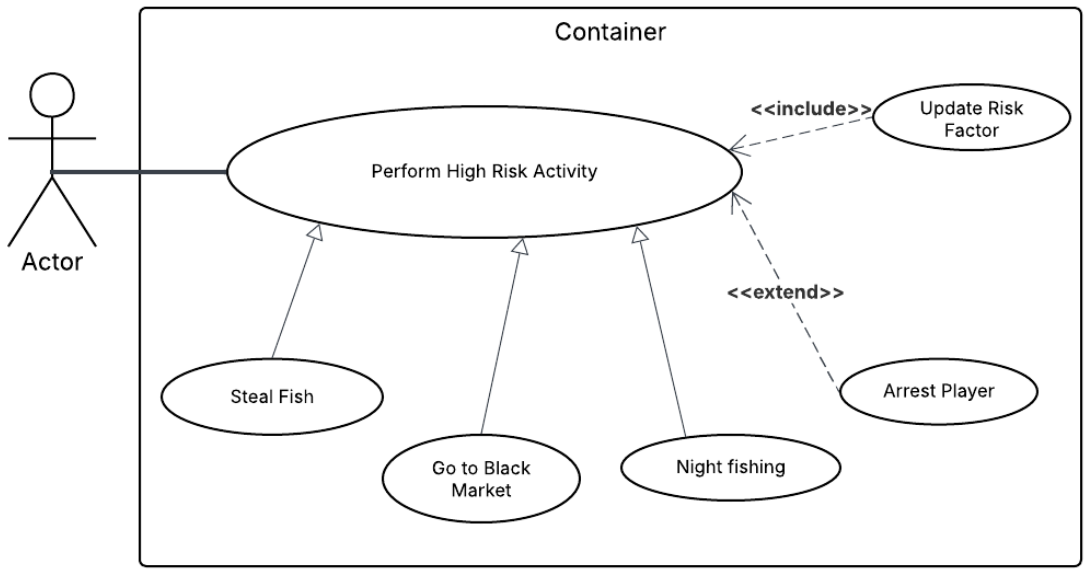
1. Brief introduction __/3

User Stats/Storyline

User stats are data that impact the gameplay. Such examples of stats would be risk factor and hunger. The storyline would be the main gameplay loop for the game.

2. Use case diagram with scenario __14

Use Case Diagrams



Scenarios

Name: Update Risk Factor

Summary: Describes how the system will adjust the risk level based on the choices made throughout the storyline.

Actors: Player.

Preconditions: The player makes a choice.

Basic sequence:

Step 1: System receives choice data.

Step 2: System calculates new risk factor data by adding the risk factor from the choice onto the current risk factor.

Step 3: System checks risk factor value to see if player can be arrested and checks for data corruption.

Step 4: System updates the UI to show the updated stat.

Exceptions:

1: Risk factor exceeds a certain amount: arrests player.

2: Risk factor data is corrupted: defaults to regular risk factor.

Post conditions: The User Stat database is updated and the new Risk Factor is displayed.

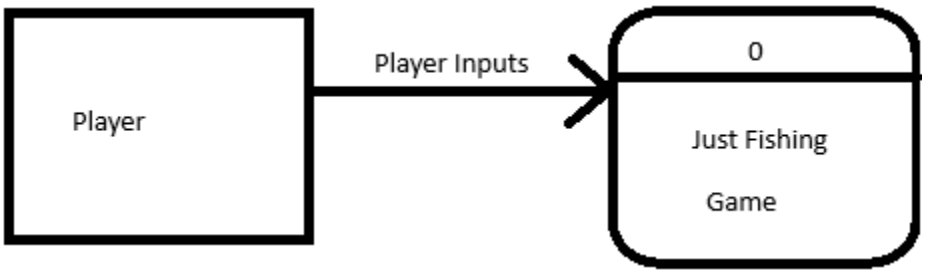
Priority: 2*

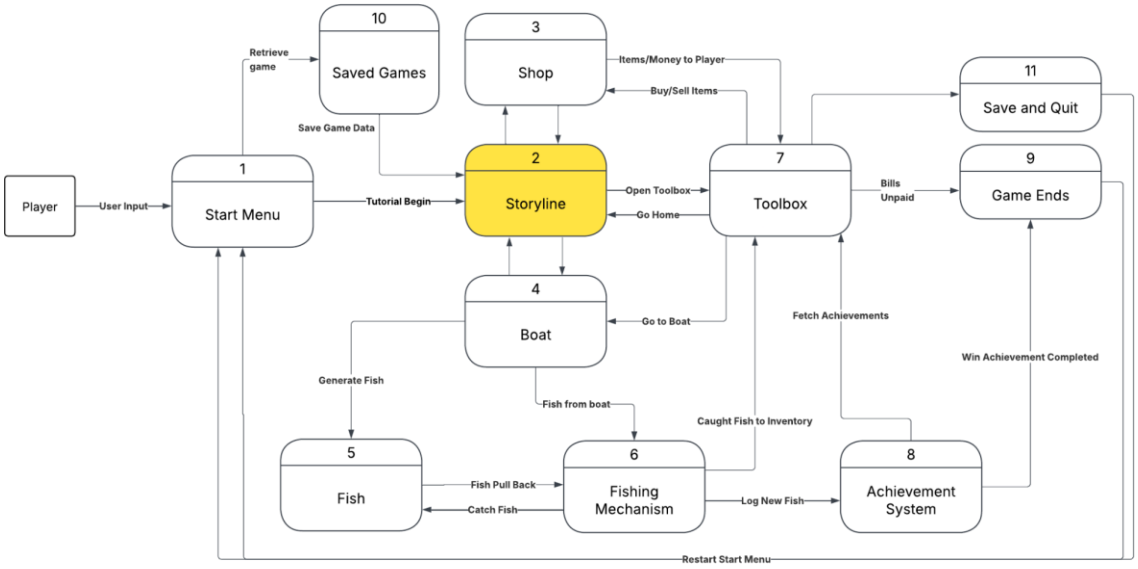
ID: C01

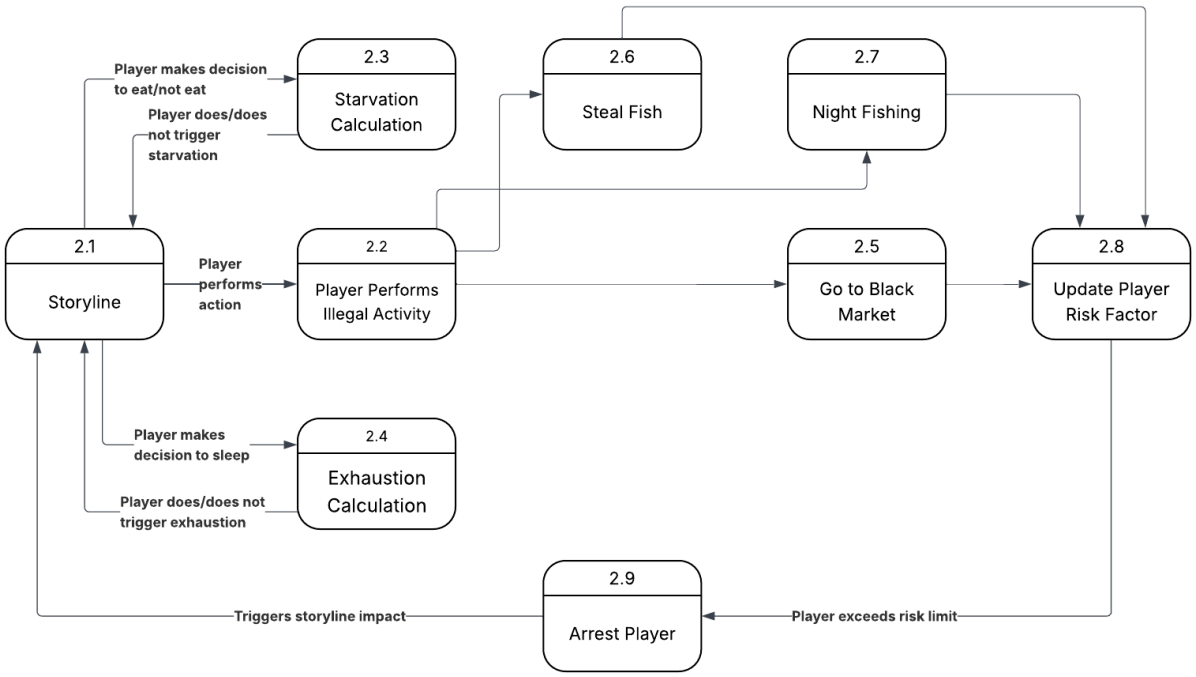
*The priorities are 1 = must have, 2 = essential, 3 = nice to have.

3. Data Flow diagram(s) from Level 0 to process description for your feature _____14

Data Flow Diagrams







Process Descriptions

Shop NPCs

```
WHILE Player is interacting with Shop NPC
  IF Player selects "Sell Fish"
    FOR EACH Fish in Player_Inventory
      IF Current_Time < 07:00 PM
        CALCULATE Fish_Value based on Rarity and Weight
        ADD Fish_Value to Player_Currency
        REMOVE Fish from Player_Inventory
        LOG Sale_Transaction to Save_Data
      ELSE
        DISPLAY "Shop is closed. All fish must be sold by 7 PM."
        EXIT Loop
      END IF
    END FOR

  ELSE IF Player selects "Buy Item"
    DISPLAY available Fishing_Upgrades (Better lures, rods, bait, etc.)
    IF Player_Currency >= Item_Price
      SUBTRACT Item_Price from Player_Currency
      ADD Selected_Item to Player_Inventory/Toolbox
      LOG Purchase to Save_Data
    ELSE
```

```
            DISPLAY "Insufficient funds."
          END IF

        ELSE IF Player selects "Black Market"
          DISPLAY Special_Items
          EXECUTE "Buy Item" Logic
          INCREASE Player_Risk_Factor
        END IF
      END WHILE
```

Dialogue/Event Generator/NPCs

```
      WHILE Player is interacting with Shop NPC
        IF Player selects "Sell Fish"
          FOR EACH Fish in Player_Inventory
            IF Current_Time < 07:00 PM
              CALCULATE Fish_Value based on Rarity and Weight
              ADD Fish_Value to Player_Currency
              REMOVE Fish from Player_Inventory
              LOG Sale_Transaction to Save_Data
            ELSE
              DISPLAY "Shop is closed. All fish must be sold by 7 PM."
              EXIT Loop
            END IF
          END FOR

        ELSE IF Player selects "Buy Item"
          DISPLAY available Fishing_Upgrades (Better lures, rods, bait, etc.)
          IF Player_Currency >= Item_Price
            SUBTRACT Item_Price from Player_Currency
            ADD Selected_Item to Player_Inventory/Toolbox
            LOG Purchase to Save_Data
          ELSE
            DISPLAY "Insufficient funds."
          END IF

        ELSE IF Player selects "Black Market"
          DISPLAY Special_Items
          EXECUTE "Buy Item" Logic
          INCREASE Player_Risk_Factor
        END IF
      END WHILE
```

Sava Data/Game Storage

```
ON RECEIVE "Read State" request
   ACCESS Game_Storage_File
   RETURN (Inventory_List, Fish_Catalogue, Achievements, Currency,
Boat_Damage, Risk_Factor, Story_Flags)

ON RECEIVE "Write State" request from external process (Shop, Fishing, or
Event)
   UPDATE Game_Storage_File with incoming data:

   IF Data contains "Caught Fish"
      UPDATE Fish_Catalogue
      IF Fish meets "Achievement" criteria
         GENERATE Achievement_Record
         ADD Special_Item reward to Inventory
      END IF


   IF Data contains "Risk Factor"
      IF Risk_Factor > Arrest_Threshold
         TRIGGER "User Arrested" Game_Over event
      END IF
   END IF

COMMIT updates to persistent storage
```

4. Acceptance Tests _____9

Risk Factor and Arrest Tests

These tests ensure that the “Arrest” exception triggers at exactly the right moment

Examples for Risk Factor and Arrest Tests

Input	Current Risk (int)	Expected Output	Notes
Steal Fish (+5 risk)	95	Risk = 95; State=Free	Normal Case: updates to persistent storage increased but remains below the threshold
Night Fish (+2 risk)	98	Risk = 100; State=Arrested	Boundary: Exact threshold reached; triggers Arrest extension

Buy Tools (+10 risk)	90	Risk=100; State=Arrested	Boundary: Risk exceeds 100; should cap at 100 and arrest
Any Action	100	State=Arrested	Bad Input: Ensure an already arrested player cannot act

Hunger and Starvation Tests

These focus on the allocation of money and the resulting hunger change.

Examples for Hunger and Starvation Tests

Input (Money for Food)	Current Hunger	Expected Output	Notes
$10 (Daily Req)	50	100 Hunger	Boundary: Minimum met; hunger is stable
$0	10	0 Hunger	Boundary: No food leads to starvation/death.
$20 (Surplus)	95	100 Hunger	Boundary: Hunger increases but caps at max (100)
-$5 (Error)	50	45 Hunger	Bad Input: System should treat negative input as $0

Sleep and Exhaustion Tests

Since not getting enough sleep in a week can kill the player, you need a test for the cumulative effect.

Examples of Sleep and Exhaustion Tests

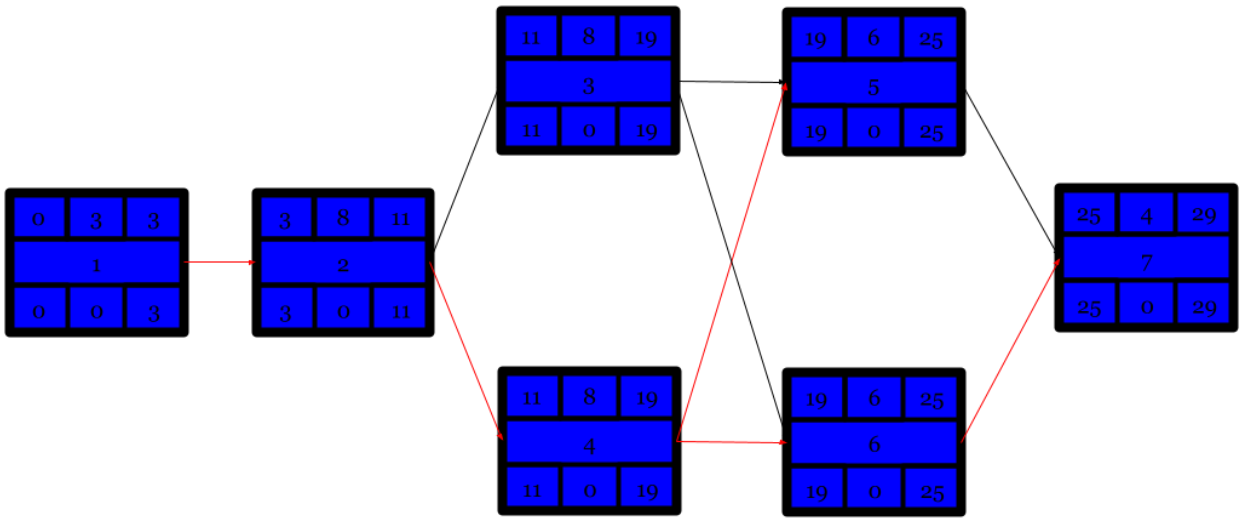
Input (Weekly Sleep Hours)	Current Energy	Expected Output	Notes
30 Hours	100	100 Energy	Normal Case: Sufficient sleep maintained
12 Hours	100	20 Energy	Penalty: Significant energy drop for low sleep.
5 Hours	15	0 Energy	Boundary: Extreme exhaustion leads to death

5. Timeline _____/10

Work items

Task	Duration (hrs)	Predecessor Task(s)
1. Requirements Finalization	3	-
2. Logic Design	8	1
3. Risk System Backend	8	2
4. Survival System Backend	8	2
5. UI & HUD Integration	6	3, 4
6. Automated Unity Testing	6	3, 4
7. Balancing & Polish	4	5,6

Pert diagram



Gantt timeline

1																
2				1												
3												2				
4												2				
5																
6																
7																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

1												
2												
3												
4												
5			3,4									
6			3,4									
7									5,6			
	18	19	20	21	22	23	24	25	26	27	28	29